

CASA: A Context-Aware Security Pipeline for Personal AI Agents

Chia-Chiao Liu | ccl@tamu.edu



Motivation

Large language model agents increasingly act as autonomous software workers: they read files, invoke tools, open network connections, and execute commands on behalf of users.

This creates a new security problem: suspicious agent behavior is often **not visible in a single event**. It emerges from a sequence of ordinary-looking actions whose meaning only becomes clear over time.

For example, a prompt injection may cause an agent to connect to a remote server, write a file, and later execute it. CASA is motivated by the need to reconstruct this kind of **behavior-level meaning** from low-level system events.

Key Challenge

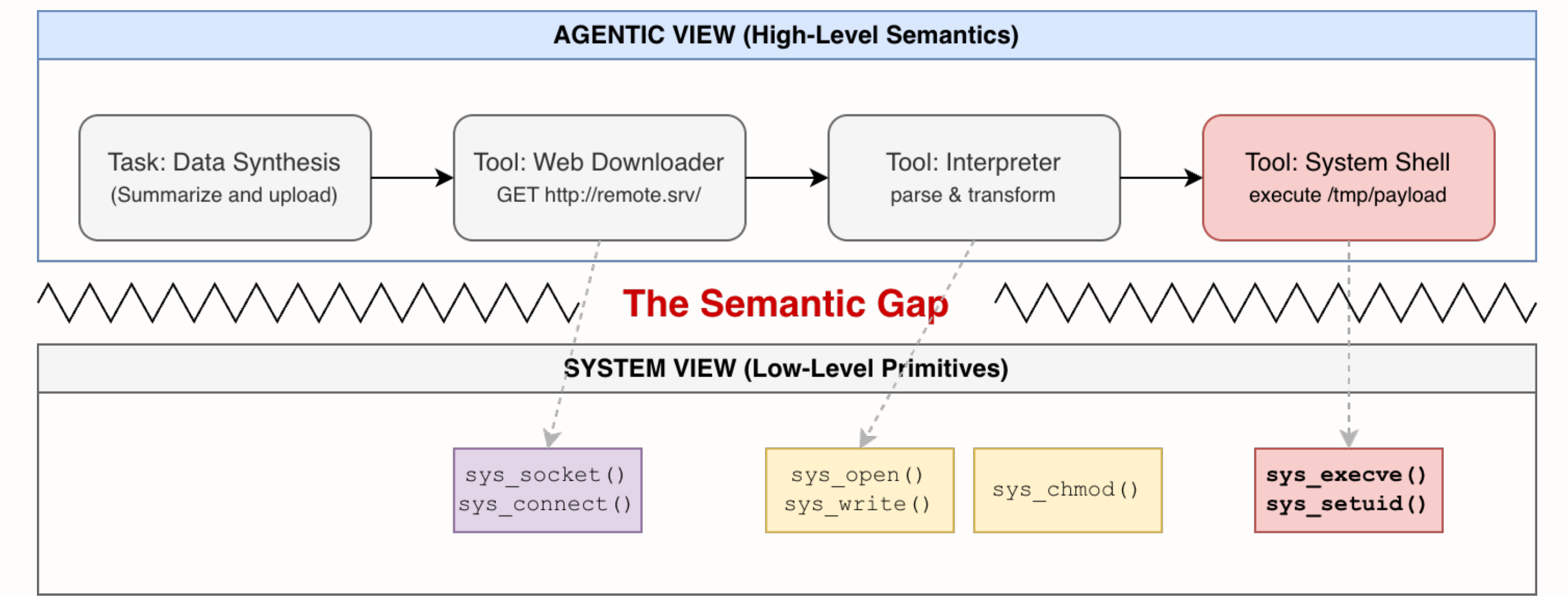
AI-agent behavior introduces a fundamental challenge for system security:

- Actions are generated dynamically at runtime
- Behavior spans multiple processes and execution steps
- Each individual event may appear benign in isolation

As a result, security decisions cannot rely on **single events**. They must reason over **how actions compose over time**.

This creates a **semantic gap** between low-level system observations and high-level agent behavior.

Example Attack Pattern



CASA targets behaviors whose meaning only becomes clear after cross-event correlation, such as **download-and-execute**, **sensitive-file access followed by exfiltration**, and **write-then-exec**. These patterns are hard to capture with isolated event inspection.

Why Existing Defenses Fall Short

Existing runtime defenses provide strong low-level visibility, but they often remain **event-centric**. They can observe system calls, but they do not naturally explain how separate events compose into agent behavior.

CASA addresses the semantic gap between:

- what the kernel observes as isolated events
- what a human recognizes as suspicious behavior
- what a policy needs to reason about across time

My Contribution

Concretely, CASA contributes:

- a **two-plane architecture** that separates eBPF-based event collection from user-space security analysis
- a **session model** that groups agent activity across processes and execution steps
- a **structured context representation** over execution, capability, and history
- a **programmable scoring engine** for multi-step suspicious behaviors
- a prototype implementation targeting **OpenClaw** as a representative personal-agent runtime

Threat Model

We consider scenarios where an AI agent is indirectly influenced through prompt injection, malicious tool outputs, or unintended instruction execution.

The attacker does not require direct system-level access. Instead, they steer the agent into system-level actions such as file access, process execution, and network communication.

In Scope

- multi-step behaviors that become meaningful only through cross-event correlation
- agent-induced system activity across processes and execution stages
- runtime detection and characterization of suspicious sessions

Out of Scope

- kernel compromise or attacks below the operating system boundary
- guaranteeing prevention before execution
- reasoning about model internals or prompt semantics in isolation

Our goal is to detect and characterize suspicious runtime behavior, rather than to prevent all possible attacks.

Design Goals

CASA is designed around three goals:

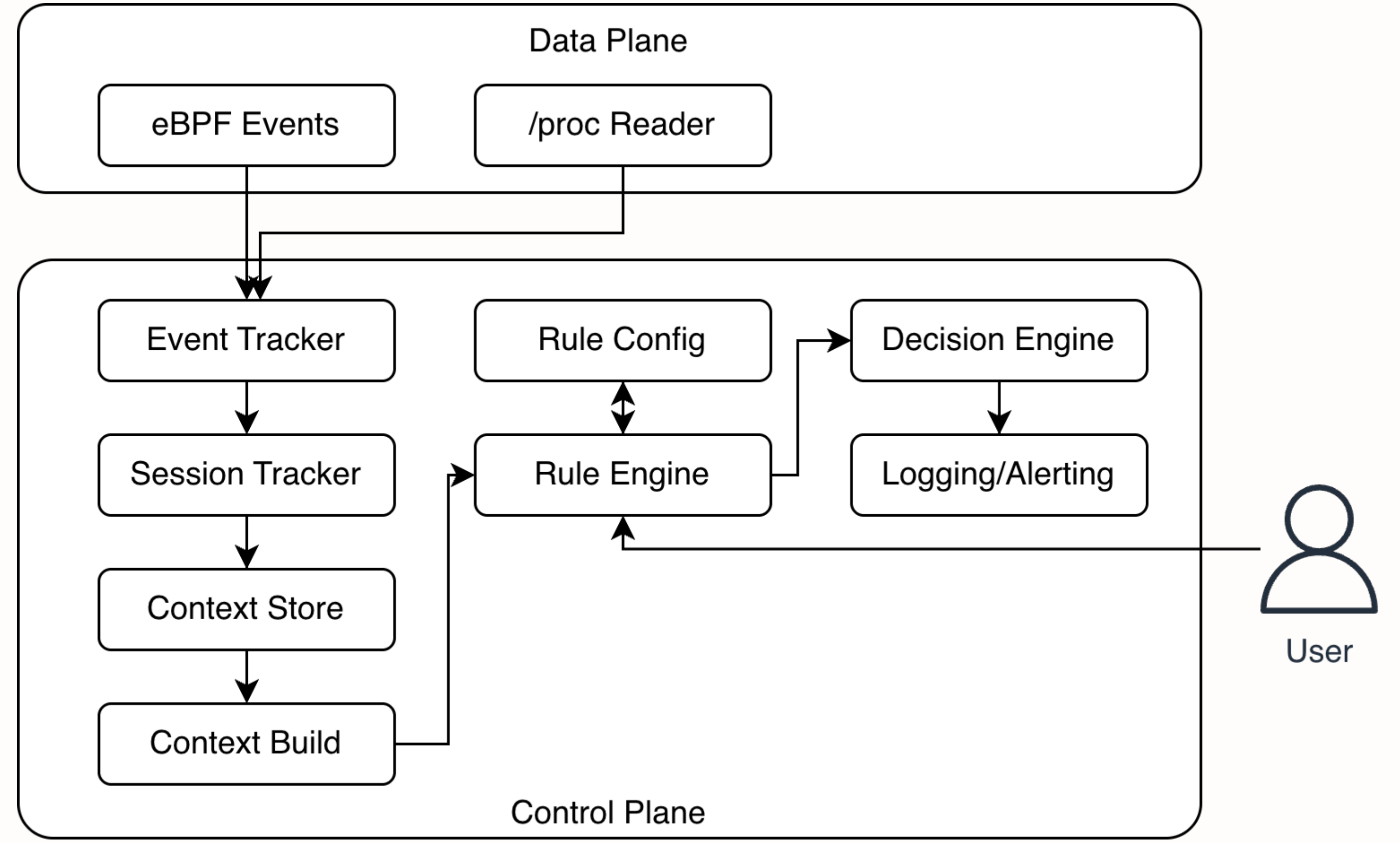
- reconstruct multi-step execution behavior rather than analyze isolated events
- remain non-invasive and agent-agnostic at the operating system boundary
- support programmable policies for flexible deployment

System Overview

CASA is organized as a **two-plane pipeline** separating data collection from analysis.

- **Data Plane: eBPF Events** capture process execution, file access, network communication, and exit events. A **/proc Reader** retrieves process-state attributes such as capabilities and seccomp mode.

- **Control Plane:** The **Event Tracker**, **Session Tracker**, **Context Store**, and **Context Builder** progressively reconstruct session-level context. The **Rule Engine** and **Decision Engine** convert derived context into risk scores and **Logging/Alerting** outputs.



Execution Flow

CASA processes system activity as a staged pipeline:

- eBPF probes capture system events such as `execve`, `openat`, and `connect`
- events are grouped into sessions based on process lineage
- session state is incrementally updated and used to derive structured context
- the rule engine evaluates context and updates a session-level risk score
- decisions are emitted when thresholds are crossed

Session-Centered Analysis

The core analysis unit is a **session**. In the current prototype, a session corresponds to **one OpenClaw CLI invocation window**. This lets CASA preserve continuity across:

- parent-child process relationships
- `execve`-based image replacement
- multi-step behavior spanning multiple binaries

This session model makes it possible to reason about how an action originated, how it evolved, and what happened before it. Instead of scoring every event independently, CASA keeps a running view of an entire agent-facing activity window.

Derived Context

CASA derives context along three dimensions:

- **Execution:** lineage depth, shell presence, network-tool presence, suspicious-path execution, deleted or memfd-backed execution
- **Capability:** dangerous capabilities, dangerous capability count, seccomp-disabled state
- **History:** connect-then-exec, sensitive-then-network, sensitive-then-exec, bursty open/connect/exec, write-then-exec on the same path

Representative fields include `execution.deep_chain`, `capability.seccomp_disabled`, and `history.connect_then_exec`.

Noise Reduction

To improve signal quality, CASA filters routine background activity before context construction, including loader behavior, helper commands, and loopback traffic.

This prevents benign events from polluting session context and inflating risk scores.

Programmable Rule Engine

Rules evaluate over **derived context rather than raw events directly**. This enables concise expression of multi-step behaviors without explicit state tracking. Each rule can trigger **at most once per session**, and only newly triggered rules contribute additional score.

A representative rule is:

```
history.connect_then_exec && execution.shell_in_chain
```

This captures a shell-mediated network-to-execution pattern. The total session risk is computed as:

$$\text{Risk}(s) = \sum_{i=1}^n w_i \cdot f_i(s)$$

Decision Outputs

CASA uses a two-threshold model:

- $\text{score} < T_1$: ignored
- $T_1 \leq \text{score} < T_2$: audit-level output
- $\text{score} \geq T_2$: alert-level output

The system produces:

- **event log:** filtered activity within the tracked process tree
- **session log:** session lifecycle snapshots
- **audit log:** lower-threshold outputs
- **alert log:** higher-threshold outputs

Each audit/alert record includes the triggering event, evaluated session context, cumulative score, and newly matched rules, providing lightweight explainability for why a session was flagged based on its behavioral context.

Evaluation Summary

CASA is evaluated with a **30-case suite** spanning **benign**, **malicious**, and **boundary** workloads, designed to exercise **multi-step behaviors** rather than isolated system calls.

Key observations:

- Benign cases are largely silent, indicating low noise under routine activity
- Alerts are triggered when high-risk patterns (e.g., write-then-execute) are actually realized
- Detection depends on realized behavior rather than prompt intent
- Boundary cases reveal ambiguity due to mixed behaviors and environment-specific noise

CASA reliably detects cross-step behaviors such as **sensitive-file** → **network**, **connect** → **exec**, and **write** → **exec** when they occur at runtime.

Runtime Overhead

CASA operates as a practical **always-on monitor** with low and stable resource usage:

- CPU usage remains low at roughly **1.2%**
- Memory usage remains stable at about **60 MB RSS**
- Internal event-to-log latency is stable at roughly **123 μs**

Overhead is primarily driven by **event intensity** rather than baseline computation.

Engineering Impact

CASA reduces the need for custom event-correlation logic by enabling concise, high-level policy specification.

- Sensitive → network: **7-line rule** vs. **1353 LoC**
- Connect → exec in shell lineage: **7-line rule** vs. **1602 LoC**
- Write → execute same path: **7-line rule** vs. **1597 LoC**

CASA shifts complexity from low-level event handling to **programmable behavioral policies**.

Future Work

The current prototype highlights several directions for improvement:

- LLM-driven execution is not always reliable, affecting behavior realization
- Provider communication and runtime environments can introduce noise
- Overhead evaluation will be extended with multi-run measurements
- Future work includes richer context extraction and automated rule generation

Takeaway

CASA turns low-level system events into **programmable behavioral security policies**, enabling detection of multi-step agent behaviors with low overhead.